BITOXIN: Preserving State Dependencies to Expose Integrity Failures in Stateful Quality Assurance

Anonymous Author(s)

Abstract

State corruption may escape quality assurance when tests discard live state before checking it, reuse an unchecked result as their expectation, or observe only values that no longer depend on the corrupted state. This paper presents BITOXIN as a test-design method that combines persistent state, dependent operations, permitted execution interruptions, and independently specified observations. The method draws on established stateful-testing and fault-observability ideas; its reference implementation is a resumable integrity probe, not a prerequisite for product adoption. We specify its detection assumptions and evaluate observation policies through 61,440 enumerated single-bit-injection trials and 120 clean controls. For faults inserted at five bit-walk boundaries, retaining state and performing a final scan detects all 15,360 faults assigned to that policy. Omitting the scan detects 7,680; reconstructing clean state detects none. A further 15,360 trials show that corruption behind an advancing verifier can escape the current pass but is detected in the next retained pass under the stated model. Native ARM64, portable-C, and sanitizer configurations produce identical outcome records. These results validate a narrow observation mechanism rather than establish field defect coverage or improved time to discovery. We also describe how to transfer the method to protected-value libraries and transaction services without deploying a Linux module.

Keywords: stateful testing; fault observability; integrity checking; fault injection; quality assurance

1 Introduction

A system can execute a large workload without exposing every incorrect intermediate state to a test oracle. An overwritten value, an early reinitialization, or an unchecked cache may separate a fault from an observable failure. This distinction matters in systems whose behavior spans scheduling, memory ownership, session lifetimes, and device transitions. Production reports on silent data corruption show why computational errors and their eventual application-level manifestations must be considered together [1].

The practical question addressed here is: *how should a QA workload preserve and observe the state whose integrity it intends to test?* Merely increasing work does not answer that question. Nor does a long sequence imply that every intermediate error affects its final result. For example, multiplication by zero can erase an earlier arithmetic error, while rebuilding an object after a transition can destroy evidence of corruption that occurred during the transition.

Bitoxin originated in an embedded QA workflow that ran an independent computation alongside system activity, including sleep/wakeup cycles. The original motivation was to expose otherwise difficult-to-observe state corruption with software-accessible operations. Historical field recollections are not used as experimental evidence in this paper. We instead reconstruct the method, provide an executable reference, and measure its observation behavior under an explicit, finite fault model.

The reusable concept is independent of execution layer. A protected-value library can carry actual objects through arithmetic and lifecycle operations; a transaction service can retain request, buffer, and response history. Neither application requires running the reference Linux module beside the product. The product's own state and its contract become the test subject.

This paper makes three bounded contributions:

1. A design contract connecting state lifetime, data dependency, allowed interruptions, and independent observations, including conditions that invalidate an inference of detection.
2. A resumable reference probe with explicit traversal and verification invariants, supported by algorithmic, host, and virtual-kernel checks.
3. Reproducible, enumerated fault experiments that separate retaining state, checking visited locations, and extending the observation horizon.

The contribution is an engineering method and its executable analysis. We do not claim to originate stateful testing, error propagation, or test oracles; establish a new universal coverage metric; or demonstrate superiority to existing QA tools.

2 Background and Positioning

Fault observation. The reachability, infection, propagation, and revealability (RIPR) model separates ex-

ecuting a faulty location from observing its consequences [2]. BITOXIN adopts that distinction as a design discipline: exercise relevant conditions, retain affected state, and specify an observation that can distinguish an error. A dependent instruction sequence alone does not satisfy all of these requirements.

Stateful and model-based testing. Wei et al. generate tests using inferred contracts to seek states not adequately exercised by an existing suite [3]. Hypothesis supports action sequences, reuse of values produced by earlier rules, comparison with a reference model, and invariants checked between steps [4]. These capabilities can host the product-level method described here. BITOXIN's emphasis is the integrity of live state across interruptions and the explicit validation of its observation horizon, rather than a new sequence-generation algorithm.

Metamorphic relations. Metamorphic testing uses relations among multiple executions when direct expected outputs are difficult to obtain [5]. Identity and round-trip computations can supply useful relations, but related executions may share faults. Our reference probe uses an identity register transformation with a direct input/output comparison and separate memory expectations. Product adaptations should combine applicable relations with known answers or independent models.

Scheduling and event order. CHESS systematically explores thread interleavings and supports reproduction of failures [6]. Event-sequence covering methods exercise specified orders among events [7]. Both are complementary to the observation structure considered here. BITOXIN permits scheduling and records test progress; it neither controls all schedules nor claims exhaustive event-order coverage. A seed identifying a memory traversal is not a replay of execution timing.

Stress and computational integrity. Stressful Application Test generates memory and I/O traffic and checks data integrity [8]. Dixit et al. discuss in-production and out-of-production strategies for detecting silent corruption at scale [1]. These are relevant operational contexts, but our experiments do not compare their detection performance. We isolate a smaller question: what happens to a known injected error when a test retains, checks, or reconstructs the state containing it?

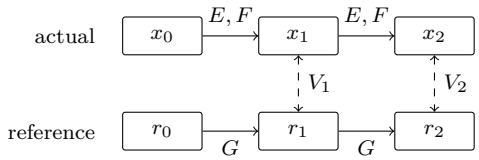


Figure 1: Actual state survives between operations. A separate reference follows intended semantics; observations compare the paths without resetting actual state.

3 A Design Contract for Stateful QA

3.1 State, events, and observations

Let x_i be the actual product state before step i, u_i its intended operation input, and e_i an event permitted by the product contract. A useful abstraction is

$$x_{i+1} = F_i(E_{e_i}(x_i), u_i), \qquad r_{i+1} = G_i(r_i, u_i, e_i), \quad (1)$$

where r_i is a separately maintained reference state. E represents an interruption or transition; G encodes its specified effect, including invalidation or rejection when applicable. An observation predicate $V_i(O_i(x_i), r_i)$ is evaluated at selected checkpoints. The oracle should not silently adopt an unchecked actual output as its new reference.

Figure 1 illustrates the two paths. The actual path retains product outputs as inputs to later operations. The reference path follows intended inputs and specified event semantics. Checkpoints connect the paths without repairing the product from the reference. A reference may be simpler than the product, but merely calling the same faulty routine twice does not make it independent.

3.2 Four requirements

Retain the relevant history. Define when a value, handle, buffer, or session is created and retired. A fault can affect an observation only if its consequences survive until that observation. Resetting between independent tests is useful for isolation; resetting inside the interval whose integrity is being tested can erase the relevant history. Both forms of testing are needed for different questions.

Carry actual dependencies. The next product operation must consume the actual preceding state. A loop that performs many unrelated computations does not create that dependency. Check for masking operations, including truncation, overwrite, zeroing, and inverse transformations that can cancel correlated errors. Longer chains need not improve detection monotonically.

Permit meaningful interruptions. Do not serialize an entire test behind a lock if its purpose is to examine

allowed interleavings. Conversely, introducing a data race forbidden by the API does not establish a product defect. Use documented synchronization and lifecycle rules. A transition may legitimately invalidate state; in that case, expected rejection is an observation, not a failure of the test design.

Specify what exposes the error. Check values, identities, lengths, generations, and required rejection outcomes as appropriate. Preserve the first mismatch and enough prefix history to reproduce it. Intermediate checkpoints can prevent later masking; an external observer is needed for hangs or resets that stop the checker itself. An assertion or typed failure is sufficient. Deliberately crashing the system is not essential to the concept.

3.3 Probability and scope

For a specified fault and a sound observation predicate, define a true detection as the conjunction of reachability R, infection I, propagation P, and revelation O. Then

$$\begin{aligned}\Pr(\text{detected}) = {} & \Pr(R)\Pr(I \mid R)\Pr(P \mid R, I) \\ & \cdot \Pr(O \mid R, I, P). \end{aligned} \tag{2}$$

This identity, when the conditionals are defined, requires no independence assumption. Data dependency does not make successive failures statistically independent. Nor does a weighted average of state-dependent success probabilities bound every individual state from below. We use this decomposition to delimit claims, not to estimate an unknown system's reliability.

The method therefore has separate activation and observation questions. A workload may increase relevant activity, or it may suppress an idle or gating condition that triggers the defect. Software fault injection can evaluate an observation mechanism without showing that the workload activates real hardware faults more often.

4 Reference Integrity Probe

4.1 Traversal and invariants

The reference engine maintains a private buffer of B bytes with $N = 8B = 2^m$ bits, an expected bit polarity, and traversal and verification cursors. Supported engine sizes range from 64 bytes to 128 MiB. A right-shift Galois LFSR visits the $N - 1$ nonzero bit indices; bit zero is visited explicitly. LFSRs provide compact deterministic sequences [9]; maximality is checked for the actual masks and convention used by this implementation.

Initially every byte is `0xff`. During a bit visit, the engine reads the containing byte, checks the selected bit against its expected polarity, carries a token containing that byte through a register transformation, checks the transformed token, and stores the byte with the selected bit inverted. After exactly N visits, it scans all B bytes for the expected uniform endpoint. Only then does it increment the completed-pass count and invert the expected polarity. The same buffer continues into the next pass.

In a fault-free execution, a pass visits each bit exactly once and the final scan confirms either all zeros or all ones. A pass costs $N + B = 9B$ logical probe operations, excluding initialization and control. The scan adds B operations to an $8B$-operation walk, or 12.5% by this accounting; this is not a measured runtime overhead. Two passes restore the initial bytes. A step budget limits work before returning control, without reinitializing state or making the steps atomic with respect to the scheduler.

4.2 Register and execution contracts

On ARM64, the token passes through a sequence of complement, rotate, and move instructions whose correct composite result is the input. The low byte is derived from the actual memory load and supplies the eventual store. Explicit input/output constraints, early-clobber output, and scratch-register clobbers express the compiler contract [10]. The implementation uses general-purpose registers and avoids reserved platform registers and floating-point state. Portable C provides a semantic reference, not equivalent physical register coverage.

The engine reports the first observed data, register-result, state, or configuration error. These labels identify checks, not root causes. A register-result mismatch does not independently identify a defective physical register. Subsequent calls preserve the latched failure.

The shared engine has a user-process runner and a Linux worker wrapper. Neither disables preemption around the computation. The kernel wrapper can cooperate with the task freezer while retaining its buffer and cursors [11]. A CPU executes no probe instructions while powered off; resumed execution inspects retained or restored state. PM notifications do not prove that a hardware sleep state was reached. These wrappers demonstrate one execution context for the method; product adoption can operate entirely in the product's existing test framework.

4.3 Checker defects found during reconstruction

Verification exposed a legacy degree-14 mask, `0x201d`, with period 126 under the implemented recurrence rather than the required 16,383. The replacement `0x3802` passes an independent linear-map order test. All 22 supported degrees, 9 through 30, satisfy the tested full-order conditions. This finding concerns a defect in the checker and must not be counted as a newly detected hardware defect.

A separate legacy hook loop performed one excess

visit. The historically used independent-worker path did not have that loop boundary. The reconstructed engine uses explicit visit counts and closure checks. These examples motivate testing the checker before attributing its reports to the environment.

5 Evaluation Design

5.1 Questions and fault model

We evaluate three questions. **RQ1:** How do state retention and a final scan affect observation of a bit flip introduced during traversal? **RQ2:** What remains unobserved when corruption occurs behind a moving verification cursor? **RQ3:** Which implementation and execution properties are supported by current checks?

The new experiments enumerate a finite software fault model. Each trial begins from a correct prefix of the real C engine with $B = 64$, $N = 512$, one of seeds $\{1, 17, 255\}$, and one of the two pass directions. Exactly one buffer bit is XOR-flipped at a call boundary. The error persists unless the tested policy overwrites it. The harness, control state, oracle, and later operations are assumed correct. Calls have a budget of one logical probe operation; no concurrent fault is injected inside a load, store, or assembly sequence.

The seeds shift the same LFSR cycle and are not independent random permutations. Enumerating them checks phase consistency, not statistical independence. We count detections, clean-control failures, and attempted post-injection calls. A call that detects a mismatch counts as one attempt. Attempt counts are operation-scale latency, not elapsed time.

The protocol and analytical expectations were written before running these experiments. Production sources were not modified for injection. The harness is linked against the shared engine and changes only its own test buffer at declared boundaries.

5.2 Experiment 1: retaining and observing state

Inject after $k \in \{0, 128, 256, 384, 512\}$ completed bit visits and enumerate every target bit. Three policies operate on separate trials:

Retain + scan: Continue with the corrupted state through the current pass's final scan.

Walk only: Retain the state but stop when the walk ends, before the scan.

Reset + replay: Reinitialize the state, reconstruct the clean prefix at the same boundary, and then finish the pass with a scan.

The last policy intentionally models loss of evidence through reconstruction; it is not a competitive QA tool. Its reconstruction cost is excluded from post-boundary attempt counts, and no performance comparison is made. Each policy has $5 \times 512 \times 3 \times 2 = 15{,}360$ injected trials and 30 clean controls.

Under this model, an unvisited corrupted bit fails its selected-bit check. A visited corrupted bit is preserved by subsequent correct updates to other bits and fails the final byte scan. Thus the predicted count per 512-bit group is 512 for Retain + scan, $512 - k$ for Walk only, and zero for Reset + replay. This argument assumes a single persistent fault between operations; it does not cover arbitrary concurrent corruption or damage to the checker.

5.3 Experiment 2: verification horizon

Complete the walk, verify $q \in \{0, 16, 32, 48, 63\}$ bytes, and inject into every bit. Continue until detection or the next pass boundary, recording separately whether the current pass detected the error. There are another 15,360 injections and 30 clean controls.

A fault in an already verified byte can escape the current scan. The predicted current-pass detection count is $512 - 8q$ per 512-bit group. If execution retains the state into the following pass, the selected-bit check eventually reaches the corrupted bit. Under the stated model, all 512 locations are therefore detected by that following boundary. This experiment makes the observation horizon explicit instead of interpreting one completed scan as perpetual integrity.

5.4 Environment and reproducibility

The new campaign ran on macOS ARM64 with Apple Clang 21.0.0, using optimized native ARM64 and forced portable-C builds, and a native AddressSanitizer/UndefinedBehaviorSanitizer build. All three produced identical raw outcome rows. These are three executions of the same finite cases, not three independent populations of defects. Sanitizers do not instrument the assembly internals.

Separate preserved verification records cover Linux ARM64 with GCC 12.2.0, a warning-enabled module build against Linux 6.1.0-53 ARM64 headers, and a QEMU 7.2.22 guest with two Cortex-A53 virtual CPUs. The virtual-kernel PM check uses `pm_test=freezer`; it does not execute physical sleep. Experiment scripts, compressed trial data, compiler records, and source hashes accompany the manuscript.

6 Results

6.1 RQ1: retention and the endpoint scan

Table 1 reports detections at each injection boundary, pooled over seeds and directions. Retain + scan detected

Table 1: Experiment 1: detections out of 3,072 injections per cell. k is the number of completed bit visits.

k	Retain + scan	Walk only	Reset + replay
0	3,072	3,072	0
128	3,072	2,304	0
256	3,072	1,536	0
384	3,072	768	0
512	3,072	0	0

Table 2: Experiment 2: detections out of 3,072 injections per row. The following-pass column includes earlier detections.

q	Current pass	By following pass	Max. attempts
0	3,072	3,072	64
16	2,304	3,072	560
32	1,536	3,072	544
48	768	3,072	528
63	48	3,072	513

15,360 of its 15,360 injected faults. Walk only detected 7,680 of 15,360, and Reset + replay detected zero of 15,360. All 90 clean controls completed without a report. Figure 2 shows the exact finite-case fractions.

The 50% aggregate for Walk only is a consequence of the chosen, equally weighted boundaries. It is not an estimated field detection probability. At the midpoint, the policy misses exactly the already visited half of the bits. The final scan provides an observation for those locations; reconstruction removes the error before either mechanism can observe it. These are mechanism checks against deliberately weakened policies, not evidence of superiority to mature memory-test tools.

6.2 RQ2: a completed pass has a finite horizon

Table 2 shows that current-pass detection declines with the number of already verified bytes. At $q = 63$, only 48 of 3,072 cases are detected before that pass completes; the other 3,024 cases involve bytes behind the cursor. All 15,360 scan-boundary faults are detected by the end of the following retained pass. The maximum observed post-injection detection latency across these cases is 560 attempted probe operations. All 30 additional clean controls complete without a report.

Across both experiments there are 61,440 injected trials and 120 clean controls per build configuration. Zero observed clean-control failures means only that these bounded controls did not falsely report a mismatch. The deterministic enumeration does not support inferential confidence intervals for hardware defect coverage.

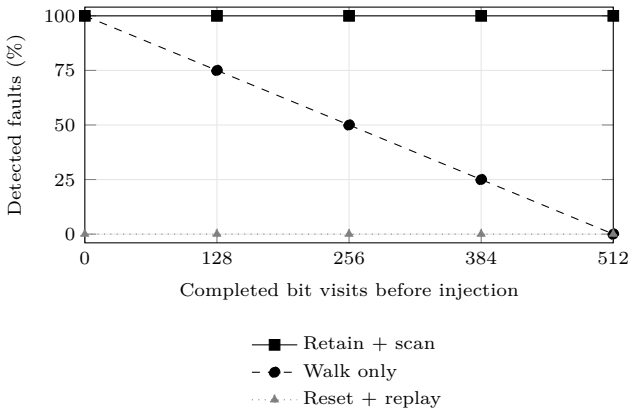


Figure 2: Observed fractions for the enumerated model. Each point pools 3,072 injections. Lines connect discrete boundaries; they are not a fitted probability model.

6.3 RQ3: implementation checks

The independent order tests cover all 22 LFSR masks. Register-lane checks compare 10,128 chosen input patterns with their expected identity outputs. Core tests cover full and inverse passes, bit zero, chunked continuation, early and late corruption, malformed cursors, invalid configuration, and latched errors. A test-only lane substitution confirms that an altered output is rejected before the corresponding buffer write. It is a simulated output error, not a physical register fault.

Native, portable, sanitizer, and Linux user-space tests pass in the recorded environments. The virtual-kernel run demonstrates bounded two-worker completion, retained cursors across freezer tests, unloading active workers, and five repeated load/unload cycles. Startup resource-failure injection and the actual panic path remain unexecuted. None of these checks establishes a physical interruption inside a particular assembly instruction, target-board qualification, or real power-sequence coverage.

7 Product Applications

The following are application designs, not evaluated integrations. The test adapter should expose product operations and observations, while an independent model represents values, ownership, and lifetimes. Sequence generation may be random, covering-based, or hand-authored. The essential requirement is that it operates on actual retained product state.

7.1 Protected-value libraries

Consider a library that represents values through protected storage and session-scoped handles. A test can create $x = 37$, add 11, copy x to y, permit an allowed scheduling boundary, multiply y by 3, and subtract 5 from x. A separate arithmetic model expects $x = 43$

and $y = 144$. The test then continues with those objects rather than reconstructing them from expected values.

Checks should exercise the protected representation, not merely a cached plaintext result. Use bounded integer arithmetic to avoid undefined overflow in the test. Across key rotation or shutdown, model the documented preservation or invalidation contract: an invalid handle should yield the specified rejection, while a state promised to survive must continue to produce the expected result. Positive controls prevent a disabled implementation from passing by rejecting every operation.

Identity relations remain useful, but a chain consisting only of zero values or inverse pairs can hide errors. Intermediate checkpoints and controlled test faults assess whether corrupted state reaches the intended verification interface. This is a library-test design; no kernel integration is implied.

7.2 Stateful transaction services

For a service using framed requests and shared buffers, carry a transaction through begin, input transfer, execute, and read operations. Check request identity, status, output length, and an independently known result. Continue with a distinct subsequent request while retaining service and buffer history so stale data and ownership mistakes remain observable.

Vary permitted fragmentation, descriptor offsets, buffer reuse, and ownership handoffs. Negative sequences require explicit expected status and post-rejection state. A timeout, abort, or reconnect need not preserve a request transparently unless the protocol promises that behavior. Fault injection belongs at defined, test-owned interfaces.

For cryptographic computation, compare deterministic byte or coefficient fixtures exactly and approximate decoded values using the profile's specified scale and tolerance. Keep independent known answers rather than relying solely on a round trip that can share an implementation error. A host simulator validates these contracts; it does not establish physical shared-memory coherency or retention across power loss.

7.3 A product-level comparative study

A subsequent evaluation should compare independent short cases, linked cases, and linked cases with permitted event perturbations. Match operation mix, operation count, resource budgets, and observation duration where possible. Separate detection of injected faults from discovery of confirmed natural defects. Record first valid failure time, unique root causes, reproduction denominators, false reports, and test overhead; retain runs with no observed failure as censored observations where time-to-event analysis is used.

When minimizing a failure, preserve the prefix that establishes the relevant state and event order. Confirm attribution by fixing the product defect and rerunning the same scenario. This study has not yet been conducted for either application design.

8 Threats to Validity

Fault-model validity. Single persistent buffer-bit flips at call boundaries are a restricted model. Real failures may be transient, correlated, multi-bit, control-flow related, or capable of corrupting the checker and its reference simultaneously. We test neither physical fault activation nor adversarial resilience. The enumerated detections cannot be generalized to all faults, instructions, or devices.

Internal validity. The same harness drives the policies, so a harness bug could affect all results. To reduce that risk, group counts are checked against independently specified arithmetic expectations, reported error locations are checked against injection locations, and outcome rows are compared across native, portable, and sanitizer configurations. The policies share the engine; the experiment is an ablation of observation behavior, not an independent reimplementation or an external-tool benchmark.

External validity. The injection buffer has only 64 bytes. It does not exercise realistic cache capacities, paging behavior, DDR traffic, or heterogeneous memory. The LFSR's exhaustive bit-index traversal is not exhaustive coverage of system states or instruction n-grams. A cache hit need not access DRAM. The virtual-kernel tests cannot reproduce electrical effects of clock gating, voltage transitions, or memory retention.

Measurement and operational validity. We report logical attempts, not timing or throughput. Checking changes the workload and may alter the very condition being investigated. More activity can prevent low-power entry. An absence of reported corruption is not a reliability certificate, and structured logs can be lost in a crash or power failure. Independent workload and transition observations are needed in real QA campaigns.

Scope of the evidence. The experiments demonstrate specified observability properties and checker behavior. They do not show faster discovery of previously unknown product defects, validate the historical embedded-system anecdote, or measure the proposed product adaptations. Establishing those claims requires external comparative experiments.

9 Conclusion

BITOXIN expresses a practical QA requirement: preserve the state whose integrity matters, carry it through meaningful dependencies and allowed interruptions, and observe it against an independent expectation. The reference probe and finite fault experiments show how reinitialization, omitted endpoint checks, and an advancing verifier change what a test can reveal. They also expose the limits of interpreting a completed check as continuing integrity. The concept can be applied within product test frameworks without adopting the original kernel execution model. Its effect on real defect discovery remains a question for controlled product and hardware campaigns.

Artifact Availability

The accompanying local artifact contains the reference engine, experiment harness, declared protocol, compressed per-trial data, build summaries, and a file-hash manifest. The manuscript source includes generated tables and plots derived from the recorded data. No public archive or persistent artifact identifier is claimed. Reproduction instructions and the separation between current evidence and proposed integrations are provided in the artifact documentation.

References

[1] Harish Dattatraya Dixit, Laura Boyle, Gautham Vunnam, Sneha Pendharkar, Matt Beadon, and Sriram Sankar. Detecting silent data corruptions in the wild. arXiv:2203.08989. https://arxiv.org/abs/2203.08989, 2022.

[2] Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. Cambridge University Press, second edition, 2017. doi:10.1017/9781316771273.

[3] Yi Wei, Hannes Roth, Carlo A. Furia, Yu Pei, Alexander Horton, Michael Steindorfer, Martin Nordio, and Bertrand Meyer. Stateful testing: Finding more errors in code and contracts. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 440–443, 2011. doi:10.1109/ASE.2011.6100094.

[4] Hypothesis contributors. Stateful tests. Hypothesis documentation. https://hypothesis.readthedocs.io/en/latest/stateful.html. Accessed October 5, 2026.

[5] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi:10.1145/3143561.

[6] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. Finding and reproducing Heisenbugs in concurrent programs. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2008. https://www.usenix.org/conference/osdi-08/finding-and-reproducing-heisenbugs-concurrent-programs.

[7] D. Richard Kuhn, James M. Higdon, James F. Lawrence, Raghu N. Kacker, and Yu Lei. Combinatorial methods for event sequence testing. In *IEEE Fifth International Conference on Software Testing, Verification and Validation*, 2012. doi:10.1109/ICST.2012.147.

[8] stressapptest contributors. Stressful application test. Project documentation. https://github.com/stressapptest/stressapptest. Accessed October 5, 2026.

[9] Peter Alfke. Efficient shift registers, LFSR counters, and long pseudo-random sequence generators. Technical Report XAPP052, Xilinx, 1996. https://docs.amd.com/v/u/en-US/xapp052.

[10] Free Software Foundation. Extended asm. GCC documentation. https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html. Accessed October 5, 2026.

[11] Linux kernel contributors. Freezing of tasks. Linux kernel documentation. https://docs.kernel.org/power/freezing-of-tasks.html. Accessed October 5, 2026.